

ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 4 OF 9

DATA ARCHITECTURE

ER Diagrams - Physical Schema - Indexing & Partitioning - RLS - Migrations

Field	Value
Document Status	Draft - v0.1 - Phase 4 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning
Upstream Dependency	Vol. 1 Business Architecture; Vol. 2 Application Architecture (bounded contexts, UML); Vol. 3 Technical Architecture (PostgreSQL, shared-schema + RLS multi-tenancy decision)
Downstream Volumes	Vol. 5 Integration Architecture; Vol. 6 Security Architecture; Vol. 7 Infrastructure & Deployment; Vol. 8 Testing & QA; Vol. 9 UI/UX & API Design

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume turns the domain model (Vol. 2 Section 8 UML), the bounded-context ownership map (Vol. 2 Section 3.2), and the PostgreSQL/shared-schema/RLS decision (Vol. 3 Section 2.2, 11) into a physical data architecture: entity-relationship diagrams per bounded context, physical table definitions for the highest-traffic entities, indexing and partitioning strategy, the concrete Row-Level Security implementation, and the schema migration and data retention strategy.

1.2 Schema Ownership Rule

Consistent with Vol. 3 Section 3 ("a module may only import another module's public export surface"), the same rule applies at the data layer: each bounded context owns a PostgreSQL schema (namespace) and is the only code allowed to write to its own tables. Other contexts read via the owning context's application-layer API (Vol. 2 Section 11), never via direct SQL joins across schema boundaries. Where a report genuinely needs cross-context joins, it queries the OpenSearch read-model (Vol. 3 Section 2.6) built from published domain events, not the transactional schemas directly.

PostgreSQL schema-per-context naming

```
identity_tenant.* -- CTX-01
ledger.*          -- CTX-02
receivables.*     -- CTX-03
payables.*        -- CTX-04
banking.*         -- CTX-05
payments.*        -- CTX-06
inventory.*       -- CTX-07
projects.*        -- CTX-08
payroll.*         -- CTX-09 (R3)
tax_compliance.*  -- CTX-10
documents.*       -- CTX-12
collaboration.*   -- CTX-13
automation.*      -- CTX-14
```

2. Schema Design Principles

#	Principle	Implementation
D1	Every table carries tenant_id and is protected by Row-Level Security	tenant_id UUID NOT NULL on every table; RLS policy per table (Section 7)
D2	The ledger is append-only	ledger.journal_entries and ledger.journal_lines have no UPDATE/DELETE grants for the application role; corrections are reversing entries (Vol. 1 Principle P4)
D3	Primary keys are UUIDv7 (time-ordered UUIDs), not sequential integers	Avoids leaking tenant record-count/growth-rate via sequential IDs; time-ordering keeps index locality reasonable despite UUID randomness
D4	Money is stored as fixed-point NUMERIC, never floating point	NUMERIC(19,4) for amounts, paired with a currency CHAR(3) column (Vol. 3 Section 4.2 CS-03 Money type)
D5	Every table has created_at, updated_at, and created_by/updated_by audit columns	Populated by application layer, indexed for audit-trail queries (Vol. 1 BC-20)
D6	Soft-delete only where the business allows deletion; financial records are never hard- or soft-deleted, only voided/reversed	is_voided flag + reversing entries for financial tables; deleted_at nullable timestamp for genuinely deletable master data (e.g. draft-only records)
D7	Foreign keys always include tenant_id in the join condition (composite FK or enforced via RLS + application check)	Prevents cross-tenant relationship bugs even if a query bypasses RLS accidentally (defense in depth)

3. Entity-Relationship Diagrams

ER diagrams are grouped by bounded context/schema, matching Section 1.2's ownership rule. Cross-schema references shown as dashed relationships are logical (enforced at the application layer + eventual consistency via domain events), not physical foreign keys, per Section 1.2.

3.1 Identity & Tenant Schema (CTX-01)

ER diagram - identity_tenant schema

```
erDiagram
    TENANT ||--o{ ORG_USER : has
    TENANT ||--o{ ROLE : defines
    ROLE ||--o{ ROLE_PERMISSION : grants
    ORG_USER ||--o{ USER_ROLE : assigned
    ROLE ||--o{ USER_ROLE : assigned
    TENANT ||--o{ EDITION_ENTITLEMENT : has

    TENANT {
        uuid id PK
        string name
        string default_currency
        string edition
        timestampz created_at
    }
    ORG_USER {
        uuid id PK
        uuid tenant_id FK
        string email
        string display_name
        boolean is_active
    }
    ROLE {
        uuid id PK
        uuid tenant_id FK
        string name
        boolean is_system_defined
    }
    USER_ROLE {
        uuid user_id FK
        uuid role_id FK
    }
```

3.2 Ledger Schema (CTX-02)

ER diagram - ledger schema

```
erDiagram
    ACCOUNT ||--o{ JOURNAL_LINE : posted_to
    JOURNAL_ENTRY ||--|{ JOURNAL_LINE : contains
    FISCAL_PERIOD ||--o{ JOURNAL_ENTRY : belongs_to
    ACCOUNT ||--o{ ACCOUNT : parent_of

    ACCOUNT {
        uuid id PK
        uuid tenant_id FK
        string code
        string name
```

```
    string account_type
    uuid parent_account_id FK
    boolean is_system_defined
}
JOURNAL_ENTRY {
    uuid id PK
    uuid tenant_id FK
    date entry_date
    string source_module
    uuid source_reference_id
    boolean is_locked
    boolean is_reversal
    uuid reverses_entry_id FK
}
JOURNAL_LINE {
    uuid id PK
    uuid tenant_id FK
    uuid journal_entry_id FK
    uuid account_id FK
    numeric debit
    numeric credit
    char currency
    numeric exchange_rate
}
FISCAL_PERIOD {
    uuid id PK
    uuid tenant_id FK
    date start_date
    date end_date
    boolean is_locked
}
```

3.3 Receivables Schema (CTX-03)

ER diagram - receivables schema

```

erDiagram
    CUSTOMER ||--o{ QUOTE : requests
    CUSTOMER ||--o{ INVOICE : billed_to
    QUOTE ||--o{ SALES_ORDER : converts_to
    SALES_ORDER ||--o{ INVOICE : converts_to
    INVOICE ||--|{ INVOICE_LINE_ITEM : contains
    INVOICE ||--o{ PAYMENT : receives
    INVOICE ||--o{ CREDIT_NOTE : offset_by

    CUSTOMER {
        uuid id PK
        uuid tenant_id FK
        string name
        string default_currency
        int payment_terms_days
    }
    INVOICE {
        uuid id PK
        uuid tenant_id FK
        uuid customer_id FK
        string invoice_number
        string status
        numeric total_amount
        numeric balance_due
        date due_date
        boolean is_recurring_generated
        uuid recurrence_rule_id
    }
    INVOICE_LINE_ITEM {
        uuid id PK
        uuid tenant_id FK
        uuid invoice_id FK
        uuid item_id
        numeric quantity
        numeric unit_price
        numeric tax_amount
    }
    PAYMENT {
        uuid id PK
        uuid tenant_id FK
        uuid invoice_id FK
        numeric amount
        date payment_date
        string method
        string idempotency_key
    }
  
```

3.4 Payables & Inventory Schemas (CTX-04, CTX-07)

ER diagram - payables and inventory schemas

```

erDiagram
    VENDOR ||--o{ PURCHASE_ORDER : receives
    PURCHASE_ORDER ||--o{ BILL : matched_to
    BILL ||--|{ BILL_LINE_ITEM : contains
    ITEM ||--o{ BILL_LINE_ITEM : references
    ITEM ||--o{ STOCK_ADJUSTMENT : adjusted_by
  
```

```
ITEM ||--o{ ITEM_BATCH : tracked_by
```

```
VENDOR {
  uuid id PK
  uuid tenant_id FK
  string name
  string default_currency
}
```

```
BILL {
  uuid id PK
  uuid tenant_id FK
  uuid vendor_id FK
  uuid purchase_order_id FK
  string status
  numeric total_amount
  string match_status
}
```

```
ITEM {
  uuid id PK
  uuid tenant_id FK
  string sku
  string name
  numeric cost_price
  numeric sale_price
  string valuation_method
}
```

```
STOCK_ADJUSTMENT {
  uuid id PK
  uuid tenant_id FK
  uuid item_id FK
  numeric quantity_delta
  string reason
}
```

```
ITEM_BATCH {
  uuid id PK
  uuid tenant_id FK
  uuid item_id FK
  string batch_number
  date expiry_date
  numeric quantity_on_hand
}
```


4. Physical Schema Design

Full DDL for all ~120 tables across 16 schemas belongs in the migration source tree, not this document. This section gives representative DDL for the three highest-risk tables (per Vol. 2 Section 8's flagged critical flows) to fix the concrete column-level conventions every other table must follow.

4.1 ledger.journal_lines (append-only, highest write volume)

DDL - ledger.journal_lines

```
CREATE TABLE ledger.journal_lines (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v7(),
  tenant_id   uuid NOT NULL REFERENCES identity_tenant.tenants(id),
  journal_entry_id uuid NOT NULL REFERENCES ledger.journal_entries(id),
  account_id  uuid NOT NULL REFERENCES ledger.accounts(id),
  debit       numeric(19,4) NOT NULL DEFAULT 0,
  credit      numeric(19,4) NOT NULL DEFAULT 0,
  currency    char(3) NOT NULL,
  exchange_rate numeric(12,6) NOT NULL DEFAULT 1,
  created_at  timestamptz NOT NULL DEFAULT now(),
  created_by  uuid NOT NULL,
  CONSTRAINT chk_one_sided CHECK (
    (debit > 0 AND credit = 0) OR (credit > 0 AND debit = 0)
  )
) PARTITION BY RANGE (created_at);

ALTER TABLE ledger.journal_lines ENABLE ROW LEVEL SECURITY;
-- No UPDATE or DELETE grants issued to the application role (Principle D2)
```

4.2 receivables.invoices

DDL - receivables.invoices

```
CREATE TABLE receivables.invoices (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v7(),
  tenant_id   uuid NOT NULL REFERENCES identity_tenant.tenants(id),
  customer_id uuid NOT NULL REFERENCES receivables.customers(id),
  invoice_number varchar(32) NOT NULL,
  status      varchar(20) NOT NULL DEFAULT 'Draft',
  total_amount numeric(19,4) NOT NULL DEFAULT 0,
  balance_due  numeric(19,4) NOT NULL DEFAULT 0,
  currency    char(3) NOT NULL,
  due_date    date,
  is_voided   boolean NOT NULL DEFAULT false,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now(),
  created_by  uuid NOT NULL,
  updated_by  uuid NOT NULL,
  UNIQUE (tenant_id, invoice_number)
);

ALTER TABLE receivables.invoices ENABLE ROW LEVEL SECURITY;
```

4.3 identity_tenant.tenants (root of tenancy)

DDL - identity_tenant.tenants

```
CREATE TABLE identity_tenant.tenants (  
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v7(),  
  name        varchar(255) NOT NULL,  
  default_currency char(3) NOT NULL,  
  edition      varchar(20) NOT NULL DEFAULT 'standard',  
  is_active    boolean NOT NULL DEFAULT true,  
  created_at   timestampz NOT NULL DEFAULT now()  
);  
-- tenants table itself has no tenant_id (it IS the tenant boundary) and no RLS policy;  
-- access to it is restricted at the application-role level instead (Vol.6 will formalize).
```

5. Indexing Strategy

Table	Index	Purpose
ledger.journal_lines	(tenant_id, account_id, created_at)	Account ledger/statement queries (most common read pattern)
ledger.journal_lines	(tenant_id, journal_entry_id)	Fetch all lines for a given journal entry
receivables.invoices	(tenant_id, status, due_date)	Aging reports, overdue-invoice job (Vol.3 Section 9)
receivables.invoices	(tenant_id, customer_id)	Customer statement / portal view
receivables.invoices	UNIQUE (tenant_id, invoice_number)	Enforce human-readable numbering uniqueness per tenant
inventory.items	(tenant_id, sku)	SKU lookup, unique constraint
banking.bank_transactions	(tenant_id, bank_account_id, transaction_date)	Reconciliation screen queries
. (all tables)	(tenant_id, id) composite or tenant_id as leading column on every index	Every query is tenant-scoped (RLS-driven); leading tenant_id keeps index scans tenant-local even under a shared table

General rule: tenant_id is always the leading column of every composite index, since RLS means every query implicitly filters on it - a non-leading tenant_id would force the planner to scan across tenants before filtering.

6. Partitioning Strategy

Only tables with unbounded per-tenant growth and high aggregate write volume across all tenants are partitioned; small reference tables are not, to avoid unnecessary operational complexity.

Table	Partition Strategy	Rationale
ledger.journal_lines	RANGE partition by created_at, monthly, with pg_partman automating new-partition creation and old-partition archival	Highest write-volume table across all tenants combined; monthly partitions keep index size and vacuum time bounded; old partitions can be moved to cheaper storage tiers per Section 10 retention policy
receivables.invoice_line_items	RANGE partition by created_at, quarterly	High volume but lower than journal_lines; quarterly balances partition count against per-partition size
banking.bank_transactions	RANGE partition by transaction_date, monthly	Continuous high-volume ingestion from bank feeds across all tenants
Audit/change-log tables (Vol.1 BC-20)	RANGE partition by created_at, monthly, with aggressive archival after the compliance retention window (Section 10)	Write-heavy, rarely read except for audits; safe to move to cold storage quickly

Explicitly NOT partitioned at Release 1: identity_tenant.tenants, ledger.accounts, receivables.customers, inventory.items - these are reference/master data with bounded per-tenant row counts (Section 11 classification).

7. Row-Level Security Implementation

Implements the Vol. 3 Section 11 decision. Every tenant-scoped table gets an identical policy shape, generated by a migration helper rather than hand-written per table, to guarantee consistency.

RLS policy template (applied per table)

```
ALTER TABLE <schema>.<table> ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON <schema>.<table>
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid)
  WITH CHECK (tenant_id = current_setting('app.current_tenant_id')::uuid);

-- Application connection sets this once per request (Vol.3 Section 11.2):
-- SET app.current_tenant_id = '8f2a1c...';
```

Safeguard	Detail
No BYPASSRLS role in application connections	The application's PostgreSQL role never has BYPASSRLS or superuser; only a narrowly-scoped migration/ops role does, used exclusively for schema changes, never application traffic
Fail-closed default	If app.current_tenant_id is not set on a connection, the policy evaluates to false for every row (no accidental full-table access) rather than failing open
Connection pooling caveat	Because RLS relies on a session-local setting, the connection pooler (e.g. PgBouncer) must run in session mode for tenant-scoped connections, or the tenant context must be re-set on every checkout in transaction-pooling mode - this is a hard requirement for Vol. 7's infrastructure sizing
Automated test	CI includes a cross-tenant-leakage test suite (Vol. 8) that attempts to read tenant B's data while scoped to tenant A and asserts zero rows returned, run against every table with RLS enabled

8. Data Types & Column Conventions

Domain Concept	PostgreSQL Type	Convention
Primary key	uuid DEFAULT uuid_generate_v7()	Time-ordered UUID (Section 2, D3)
Money amount	numeric(19,4)	Paired with a char(3) currency column; never numeric without explicit precision, never float/double
Currency code	char(3)	ISO 4217 (USD, EUR, INR, ...)
Status/enum fields	varchar with a CHECK constraint (not native PG enum)	varchar chosen over native enum so new statuses can be added without a blocking ALTER TYPE migration
Timestamps	timestamptz	Always timezone-aware; never naive timestamp
Dates without time (e.g. due_date)	date	Used only where the business concept has no time component
Audit columns	created_at, updated_at timestamptz; created_by, updated_by uuid	Present on every table; updated_at maintained by a trigger, not application code, to guarantee correctness
Soft-delete flag	deleted_at timestamptz NULL (nullable = not deleted)	Only on genuinely deletable master data (Section 2, D6); never on financial transaction tables
JSON/custom fields	jsonb	Backs Vol.1 BC-17 custom fields; indexed with a GIN index where queried

9. Migration Strategy

9.1 Tooling & Process

- Schema migrations are managed with a code-first migration tool (e.g. node-pg-migrate or Prisma Migrate, consistent with the TypeScript stack in Vol. 3) with every migration checked into the same monorepo as the module it belongs to (Vol. 3 Section 3.2).
- Because the platform uses a single shared schema per environment (not per-tenant databases, Vol. 3 Section 11), a migration runs once per environment and applies to all tenants simultaneously - this makes migration safety a first-class operational concern, not an edge case.
- All migrations must be backward-compatible with the currently-deployed application version for at least one release cycle (expand/contract pattern): add new columns as nullable or with defaults first, deploy code that writes to both old and new shapes, backfill, then drop the old column in a later migration.
- Destructive migrations (DROP COLUMN, DROP TABLE) require a two-step process: mark deprecated in one release, physically remove in the next, with a manual sign-off gate (Vol. 8 CI pipeline).

9.2 Zero-Downtime Migration Pattern

Expand/contract migration flow

```

flowchart LR
  A[Release N: Add new nullable column/table] --> B[Deploy app code that writes both old + new shape]
  B --> C[Backfill existing rows - batched, rate-limited background job]
  C --> D[Deploy app code that reads only from new shape]
  D --> E[Release N+1: Migration drops old column/table]

```

9.3 Large-Table Migration Safety (journal_lines, invoice_line_items)

Because these tables are partitioned (Section 6), schema changes are applied per-partition in small batches rather than as a single blocking ALTER TABLE across the entire (potentially billions-of-rows) table - avoiding long-held locks that would stall the append-only ledger writes financial correctness depends on.

10. Data Retention & Archival

Data Category	Retention Policy	Archival Mechanism
Financial ledger data (journal_entries, journal_lines)	Retained indefinitely online for the tenant's active subscription; statutory minimum 7 years even after tenant offboarding (varies by jurisdiction - Vol.5/6 to confirm per-country requirements)	Partitions older than 24 months moved to cheaper storage tier (Section 6); still queryable, just slower
Transactional documents (invoices, bills)	Same as ledger - these are the source records for statutory retention	Same partition-tiering approach
Audit trail / change log	Minimum 7 years per common financial-audit regulatory expectation (to be confirmed per market in Vol.6)	Aggressive partition archival to cold object storage after 12 months
Bank feed raw transaction imports	3 years online, then archived (post-reconciliation, raw import data has diminishing operational value)	Cold storage archival
Session/auth data (Redis, Vol.3 Section 7)	TTL-bound, not part of durable retention	N/A - ephemeral by design
Tenant deletion (offboarding)	Soft-deactivate tenant immediately; hard-delete only after the statutory retention window and an explicit data-deletion request (GDPR right-to-erasure interacts with financial retention law - Vol.6 to resolve the precedence)	Export-then-purge workflow, logged for compliance

11. Master Data vs. Transactional Data Classification

This classification drives both the partitioning decisions (Section 6) and the caching strategy (Vol. 3 Section 7: master data is cache-friendly, transactional data generally is not).

Classification	Examples	Characteristics
Master/reference data	tenants, accounts (chart of accounts), customers, vendors, items, tax_rates, roles	Low write frequency, bounded row count per tenant, high read frequency, cache-friendly (Vol.3 Section 7.1)
Transactional data	journal_entries, journal_lines, invoices, invoice_line_items, bills, payments, bank_transactions, stock_adjustments	High write frequency, unbounded growth over time, partitioned (Section 6), generally not cached beyond short-TTL aggregates
Derived/read-model data	OpenSearch report indices (Vol.3 Section 2.6), dashboard aggregate cache	Rebuildable from transactional data + events; not a source of truth, never backed up independently of its source events

12. Traceability & Next Phase

12.1 Traceability to Volumes 1-3

Vol. 4 Artifact	Traces To
Section 3 ER Diagrams	Vol. 2 Section 8 UML Domain Model
Section 1.2 Schema-per-context ownership	Vol. 2 Section 3.2 Bounded Context Catalog; Vol. 3 Section 3 module boundary rule
Section 7 Row-Level Security	Vol. 3 Section 11 Multi-Tenancy Implementation Strategy
Section 2 D2 Append-only ledger	Vol. 1 Principle P4 (Auditability)
Section 6 Partitioning	Vol. 3 Section 12 Performance & Scalability Targets
Section 10 Retention	Vol. 1 Section 10 Compliance & Regulatory Scope

12.2 Open Questions for Sign-off

1. Confirm statutory retention periods per target country (US/India/EU per Vol.1 Section 13.1 assumption) so Section 10's retention table can be finalized with exact durations rather than placeholders.
2. Confirm whether PgBouncer session-mode pooling (Section 7 caveat) is acceptable from a connection-scaling standpoint, or whether an alternative RLS-compatible pooling strategy is needed - this materially affects Vol. 7 infrastructure sizing.
3. Confirm migration tooling preference (node-pg-migrate vs Prisma Migrate vs other) if the team has existing expertise to weigh in, since this affects Section 9's tooling choice.

12.3 Next Phase

On approval, Phase 5 begins: Integration Architecture (Volume 5) - detailed design for banking/bank-feed integration, payment gateway integration, tax/e-filing integration, and e-commerce channel sync, all consuming the schemas and event contracts fixed in this volume and Volume 2.